

DataLens AI

Comprehensive Module Reference

Every module in `modules/` – libraries used, algorithms, formulas, parameters, outputs, and the concrete prediction for the showcase dataset.

Showcase target:

`demo_datasets/showcase_v3_full.csv` (10,110 rows × 32 columns; binary target `defaulted`; default rate ≈8.8%; 45 seeded outliers across the loan-amount / payment trio; 110 duplicate rows; 3 years of daily `application_date`; two near-perfect VIF pairs; `default_notice_date` as a deliberate target-leakage column; `customer_feedback` as a free-text column).

How to read each module entry. Every module is documented with the same four boxes: **Libraries** (every external package and the specific function imported), **Algorithm** (numbered steps), **Math** (formulas and definitions), **Output schema** (what the JSON looks like), followed by the showcase prediction.

Contents

1	Glossary – metrics, tests, and algorithms used downstream	2
1.1	Regression metrics	2
1.2	Classification metrics	2
1.3	Cross-validation	2
1.4	Distribution and normality tests	2
1.5	Outlier detection rules	3
1.6	Correlation and association	3
1.7	Stationarity tests (time-series)	4
1.8	Drift metrics	4
1.9	Regression diagnostics for ML features	4
1.10	Explainability	4
1.11	Information-theory and embeddings (for text / RAG)	5
1.12	Common abbreviations used in the document	5
2	Orchestration & I/O	6
2.1	<code>pipeline.py</code> [phase orchestrator]	6
2.2	<code>loader.py</code> [file ingestion]	6
2.3	<code>security.py</code> [guardrails]	7
2.4	<code>frontend_api.py</code> [dashboard payload assembler]	7
3	Profiling & Signals	7
3.1	<code>profiler.py</code> [shape, dtypes, summaries]	7
3.2	<code>column_roles.py</code> [multi-label semantic roles]	8
3.3	<code>dataset_signals.py</code> [boolean + numeric flags]	9
3.4	<code>analysis_inventory.py</code> [declarative registry]	9
3.5	<code>analysis_selector.py</code> [signal-driven router]	9
3.6	<code>advanced_indicators.py</code> [utility / fairness / freshness]	9

3.7	<code>signal_engine.py</code> [display cards]	10
4	Quality Scoring	10
4.1	<code>health_score.py</code> [0–100 quality grade]	10
4.2	<code>ml_readiness.py</code> [ready-for-modelling score]	10
5	Statistics	12
5.1	<code>deep_statistics.py</code> [baseline stats, legacy v1]	12
5.2	<code>deep_statistics_v2.py</code> [research-grade battery]	12
6	Anomaly Detection	13
6.1	<code>anomaly_ensemble.py</code> [6+ detectors, mean ensemble]	13
6.2	<code>advanced_indicators.py::calculate_anomalies</code> [lightweight IQR pass]	13
7	Relationships	13
7.1	<code>multicollinearity.py</code> [VIF + redundancy groups]	14
7.2	<code>target_leakage.py</code> [target-aware feature audit]	14
7.3	<code>target_analyzer.py</code> [target shape + warnings]	14
8	ML Lab	15
8.1	<code>ml_preprocessing.py</code> [shared preprocessor]	15
8.2	<code>model_leaderboard.py</code> [CV scoreboard]	15
8.3	<code>baseline_model.py</code> [model vs dummy]	16
8.4	<code>explainability.py</code> [SHAP on the winner]	16
8.5	<code>model_diagnostics.py</code> [secondary RF audit]	17
8.6	<code>feature_importance.py</code> [impurity + mutual info]	17
8.7	<code>segment_analysis.py</code> [group-wise behaviour]	18
9	Time, Text, Drift	19
9.1	<code>time_series.py</code> [auto-detect → STL → forecast]	19
9.2	<code>text_profile.py</code> [TF-IDF + LSA preview]	19
9.3	<code>drift_analysis.py</code> [compare two dataframes]	20
10	Cleaning & Visuals	21
10.1	<code>cleaner.py</code> [cleaning pass + before/after]	21
10.2	<code>chart_planner.py</code> [decides which charts to render]	21
10.3	<code>visualizer.py</code> [renders the chart specs]	21
11	AI & RAG Layer	22
11.1	<code>ai_insights.py</code> [report writer]	22
11.2	<code>agent_brief.py</code> [compact dataset packing]	22
11.3	<code>agent_tools.py</code> [tool registry]	22
11.4	<code>ai_agent.py</code> [planner / executor / critic / writer]	22
11.5	<code>rag_index.py</code> [fact index + retrieval]	23
11.6	<code>safe_pandas.py</code> [AST-allowlist sandbox]	23
12	Reports	24
12.1	<code>report_generator.py</code> [thin wrapper]	24
12.2	<code>pdf_report_builder.py</code> [editorial-dark PDF]	24
13	Library inventory and dependency map	25
14	Expected end-to-end run on showcase_v3_full.csv	25

1 Glossary – metrics, tests, and algorithms used downstream

The dashboard surfaces a lot of statistics. The definitions live here once, so module entries can refer to them without repeating themselves.

1.1 Regression metrics

R^2 (coefficient of determination). Fraction of variance in the target explained by the model.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

$R^2 = 1$ is perfect, $R^2 = 0$ matches a constant-mean predictor, $R^2 < 0$ means the model is worse than predicting the mean.

MAE – Mean Absolute Error. $\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$. Same units as the target. Robust to outliers.

MSE – Mean Squared Error. $\text{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$. Penalises large errors more than small ones.

RMSE. $\text{RMSE} = \sqrt{\text{MSE}}$. Same units as the target.

Residuals. $r_i = y_i - \hat{y}_i$. The dashboard reports `mean`, `median`, `std`, `min`, `max` on residuals; a near-zero mean and symmetric tails indicate an unbiased model.

1.2 Classification metrics

Accuracy. Fraction of predictions that are correct. Misleading on imbalanced classes (a 91%-negative dataset trivially scores 0.91 by always predicting "no default").

Precision, recall, F_1 (per class).

$$P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R}.$$

Weighted F1 / precision / recall. Per-class score weighted by class support. The default ranking metric for the leaderboard on classification tasks.

Confusion matrix. A $K \times K$ table of true class (rows) vs predicted class (columns). Diagonal = correct predictions.

Brier score. Mean squared error on predicted probabilities for a binary target: $\text{Brier} = \frac{1}{n} \sum_i (p_i - y_i)^2$. Lower is better; 0 is perfectly calibrated, 0.25 corresponds to random guessing.

1.3 Cross-validation

KFold ($K = 5$). Splits the data into 5 equal folds; trains on 4, tests on the held-out 1, rotates 5 times. Reports mean and standard deviation of the metric.

StratifiedKFold. KFold that preserves the class distribution in each fold. Used for classification. The number of splits is automatically capped at $\min(K, \text{smallest class size})$.

Holdout. A single 75%/25% train/test split. Used by diagnostics modules for confusion matrices, residuals, Brier score.

1.4 Distribution and normality tests

Skewness. Third standardized moment. $|\text{skew}| < 0.5$ symmetric, $0.5-1$ moderate, > 1 highly skewed.

Kurtosis (excess). Fourth standardized moment minus 3. > 1 heavy-tailed, < -1 light-tailed, otherwise near-normal.

Shapiro–Wilk. Most powerful normality test for small samples ($n \leq 5000$). $p < 0.05$ rejects normality.

D’Agostino–Pearson. Combines skew + kurtosis tests; works well for $n \geq 20$.

Anderson–Darling. Statistic compared against critical values at $\{15, 10, 5, 2.5, 1\}\%$ significance levels. The dashboard reports the 5% critical value: if statistic $>$ critical, reject normality.

Best-fit distribution. For each candidate distribution (Normal, Lognormal, Exponential, Gamma, Weibull, Beta) fit by MLE, then rank by $AIC = 2k - 2 \ln \hat{L}$, where k is the number of parameters and $\hat{L}$ is the maximised likelihood. Lower AIC wins.

1.5 Outlier detection rules

IQR (Tukey’s fences). Define Q_1, Q_3 , $IQR = Q_3 - Q_1$. Outlier if value is below $Q_1 - 1.5 IQR$ or above $Q_3 + 1.5 IQR$.

Classical z -score. $z_i = (x_i - \bar{x})/\sigma$; $|z| > 3$ flagged.

Robust (MAD) z . Uses median and median absolute deviation: $z_i^* = (x_i - \text{median})/(1.4826 \cdot \text{MAD})$; $|z^*| > 3.5$ flagged. The constant 1.4826 makes the MAD a consistent estimator of σ for normal data.

Mahalanobis distance. $D^2 = (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$. Captures multivariate outliers using the inverse covariance matrix; estimated robustly via the Minimum Covariance Determinant.

Isolation Forest. Builds random binary trees; anomalies have shorter average path length to a leaf because they are easier to isolate. Score $\in [0, 1]$, higher is more anomalous.

Local Outlier Factor (LOF). Density-based – compares each row’s local reachability density to its k -nearest neighbours. Score > 1 means locally less dense, i.e. more anomalous.

Elliptic Envelope. Fits a robust Gaussian; flags rows in the tails of the implied ellipse.

ECOD / COPOD. Empirical / copula-based outlier detection from pyod. Distribution-free, no neighbour search needed.

1.6 Correlation and association

Pearson r . Linear correlation $\in [-1, 1]$. $r = \text{Cov}(X, Y)/(\sigma_X \sigma_Y)$. Sensitive to outliers.

Spearman ρ . Pearson on ranks; monotonic correlation.

Kendall τ . Probability of concordant minus discordant pairs.

p -value. Probability of seeing a correlation at least this extreme if the true correlation were 0. $p < 0.05$ is the default "significant" threshold.

Chi-square test of independence. For two categorical variables, $\chi^2 = \sum(\text{observed} - \text{expected})^2/\text{expected}$ on the contingency table.

Cramer’s V . Categorical-categorical association strength derived from χ^2 : $V = \sqrt{\chi^2/(n \cdot (\min(r, c) - 1))}$, $\in [0, 1]$. Buckets: ≥ 0.5 Strong, ≥ 0.3 Moderate, ≥ 0.1 Weak.

Point-biserial r . Pearson correlation between a binary variable (coded 0/1) and a continuous one. Same scale as Pearson.

Mann–Whitney U . Rank-based test that two independent samples come from the same distribution. Non-parametric version of the two-sample t -test.

Kruskal–Wallis H . Mann–Whitney generalised to ≥ 3 groups.

1.7 Stationarity tests (time-series)

ADF (Augmented Dickey–Fuller). H_0 : a unit root is present (non-stationary). $p < 0.05$ rejects H_0 , i.e. the series is stationary.

KPSS. H_0 : the series is (trend-)stationary. $p > 0.05$ fails to reject stationarity. Combining ADF and KPSS gives one of four verdicts: stationary, non-stationary, trend-stationary, difference-stationary.

ACF / PACF. Autocorrelation function and partial autocorrelation function up to lag $\min(40, n/5)$. Slow-decaying ACF signals non-stationarity; spikes at lag k in the PACF suggest an AR(k) model.

STL decomposition. Seasonal-Trend decomposition using LOESS. Splits the series into trend T_t , seasonal S_t , and residual R_t components. Strength scores (Hyndman, 2008) are

$$F_T = \max\left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(T_t + R_t)}\right), \quad F_S = \max\left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(S_t + R_t)}\right).$$

Holt–Winters (additive). Triple exponential smoothing with trend + seasonality. Compared against a "last-value" naive baseline using MAE and RMSE on the held-out tail.

1.8 Drift metrics

PSI – Population Stability Index. For two distributions binned into B buckets,

$$\text{PSI} = \sum_{b=1}^B (p_b^{(\text{cur})} - p_b^{(\text{ref})}) \ln \frac{p_b^{(\text{cur})}}{p_b^{(\text{ref})}}.$$

Buckets: < 0.10 stable, < 0.25 minor, < 0.50 moderate, ≥ 0.50 severe.

Kolmogorov–Smirnov two-sample. Maximum absolute difference between the two empirical CDFs. $p < 0.01$ flags a real distribution shift.

Chi-square (drift). Same as above but applied to the count contingency table of categorical proportions in the two halves.

Mean z-shift. $(\bar{x}_{\text{cur}} - \bar{x}_{\text{ref}})/\sigma_{\text{pooled}}$. A simple "how many SDs has the mean moved?" view.

1.9 Regression diagnostics for ML features

VIF – Variance Inflation Factor. For feature j regressed on all others, $\text{VIF}_j = 1/(1 - R_j^2)$. $\text{VIF} \geq 10$ is high multicollinearity; ≥ 5 is moderate.

Mutual information. Information-theoretic dependency, ≥ 0 , not bounded above. Captures both linear and non-linear relationships.

Permutation importance. Drop in the model's score when a single feature's values are randomly permuted. Run over n repeats; mean + standard deviation reported.

1.10 Explainability

SHAP value. Per row, per feature contribution to the model's output, derived from cooperative game theory (Shapley values). Sums to $\hat{y}$ – baseline. Mean absolute SHAP across rows gives global feature importance; the per-row vector tells the story for one prediction.

Tree SHAP. Exact, polynomial-time SHAP for tree ensembles (Random Forest, Gradient Boosting, XGBoost, LightGBM).

Linear SHAP. Closed-form SHAP for linear models given a background sample.

1.11 Information-theory and embeddings (for text / RAG)

Shannon entropy. For category proportions p_i , $H = -\sum_i p_i \log_2 p_i$. Maximum is $\log_2 K$ (uniform). Entropy ratio is $H/\log_2 K$, a value in $[0, 1]$ where 1 is "perfectly uniform".

TF-IDF. Term frequency divided by inverse document frequency. Down-weights terms that appear in many documents.

TruncatedSVD (LSA). Truncated singular value decomposition on the TF-IDF matrix to project documents into a low-dimensional latent space (2 components for the dashboard scatter).

PSI vs KL divergence. PSI is symmetric: $\sum_b (p_b - q_b) \ln(p_b/q_b)$ vs KL $\sum_b p_b \ln(p_b/q_b)$. Both become large when the distributions diverge, but PSI is finite when both sides have non-zero coverage.

1.12 Common abbreviations used in the document

IQR Interquartile range, $Q_3 - Q_1$
MAD Median absolute deviation
AIC Akaike Information Criterion
ADF Augmented Dickey–Fuller
KPSS Kwiatkowski–Phillips–Schmidt–Shin
STL Seasonal-Trend decomposition using LOESS
PSI Population Stability Index
KS Kolmogorov–Smirnov
LOF Local Outlier Factor
ECOD Empirical Cumulative Outlier Detection
COPOD Copula-based Outlier Detection
TF-IDF Term Frequency – Inverse Document Frequency
LSA Latent Semantic Analysis (TruncatedSVD on TF-IDF)
SHAP SHapley Additive exPlanations
VIF Variance Inflation Factor
TTR Type-Token Ratio (vocab size / total tokens)
HW Holt–Winters exponential smoothing
RAG Retrieval-Augmented Generation

2 Orchestration & I/O

2.1 pipeline.py [phase orchestrator]

Single entry point that runs the entire analysis. Phase boundaries are explicit so failures in one phase cannot poison the others.

Libraries & key functions

- `concurrent.futures.ThreadPoolExecutor` – parallel phase 3.
- `logging` – structured pipeline logs.
- `time.perf_counter` – millisecond timings per phase.
- Internal: every other module in `modules/`.

Algorithm

1. **Phase 1 – Structural.** `load_dataset` → `build_profile` → `infer_column_roles` → `summarize_roles` → `detect_dataset_signals` → `select_analyses`.
2. **Phase 2 – Quality.** `calculate_health_score` → `calculate_ml_readiness` → `calculate_advanced_indicators`.
3. **Phase 3 – Parallel analytics** (`mode` ∈ {`standard`, `deep`, `research`}). Wrapped in `_safe_call`: `deep_statistics_v2`, `anomaly_ensemble`, `time-series`, `text profile`. Default `parallel_workers=4`.
4. **Phase 4 – ML chain** (only if `target_column` given). `run_model_leaderboard` → `explain_winner`.
5. **Phase 5 – Display + cleaning.** `build_signals` → `create_cleaned_dataset` → `generate_charts`.
6. **Phase 6 – AI report.** Deferred unless `DATALENS_ANALYZE_AI=1`. The dashboard fetches it on demand through `/api/ai-report`.

Every parallel branch is wrapped so a single exception turns into `\{available: false, error: ... \}` instead of sinking the run.

Output schema

Single dict with: `dataset_id`, `filename`, `analysis_mode`, `profile`, `column_roles`, `roles_summary`, `dataset_signals`, `analysis_selection`, `health`, `signals`, `ml_readiness`, `advanced`, `deep_statistics_v2`, `anomalies_v2`, `model_leaderboard`, `explainability`, `time_series`, `text_profile`, `cleaning`, `charts`, `ai_report`, and a `timings_ms` map keyed by phase.

On `showcase_v3_full.csv`: every phase fires green. `timings_ms.total` typically lands in 25–45 s on a laptop because phase 3 alone spawns four threads doing real work and the leaderboard runs CV on 5–7 models.

2.2 loader.py [file ingestion]

Reads the uploaded CSV / TSV / XLSX / XLS / JSON. Tolerates real-world encodings.

Libraries & key functions

- `pandas.read_csv`, `read_excel`, `read_json`.
- `pathlib.Path`, `uuid.uuid4` for the dataset id.
- Internal `security.validate_file` + `security.make_safe_filename`.

Algorithm

For CSV/TSV reads, an encoding waterfall is tried until one succeeds: `utf-8-sig` → `windows-1252` → `latin-1`. If all three fail, a final `latin-1` read with `errors='replace'` guarantees the file loads, possibly with garbled non-ASCII characters.

On `showcase_v3_full.csv`: loads cleanly as UTF-8; returns a (10110, 32) dataframe with no encoding fallback fired.

2.3 security.py [guardrails]**Libraries & key functions**

`werkzeug.utils.secure_filename`, plus a small extension allow-list.

Algorithm

1. `validate_file(filename)` rejects anything outside `{.csv, .xlsx, .xls, .tsv, .json}`.
2. `make_safe_filename(name)` strips path separators and suspicious characters.
3. `sanitize_csv_value(v)` prefixes a single quote to any cell starting with `=`, `+`, `-` or `@` – prevents CSV-injection attacks when the cleaned file is opened in Excel.

On `showcase_v3_full.csv`: no rejections; the cleaned export keeps every row.

2.4 frontend_api.py [dashboard payload assembler]

Wraps the pipeline result into the JSON shape the React / Streamlit dashboards consume.

Libraries & key functions

`pandas`, `numpy`, `math`; internal helpers from `column_roles`, `deep_statistics`, `target_analyzer`, `feature_importance`, `baseline_model`, `target_leakage`, `multicollinearity`, `model_diagnostics`, `segment_analysis`.

Algorithm

1. Picks the most meaningful numeric column (or first categorical) and builds a histogram preview with `min(12, max(6,  $\sqrt{n}$ ))` bins.
2. Re-runs target-aware analyses on demand (`run_feature_importance`, `run_baseline_model`, `run_target_leakage_analysis`, `run_model_diagnostics`).
3. Stitches everything into a camelCase payload with KPI tiles, download links, signals, charts, and timings.

On `showcase_v3_full.csv`: ~2–3 MB JSON payload covering every dashboard tab.

3 Profiling & Signals**3.1 profiler.py [shape, dtypes, summaries]****Libraries & key functions**

`pandas` (`describe`, `value_counts`, `corr`, `memory_usage`), `numpy` for NaN handling.

Algorithm

1. Detects dtypes via `select_dtypes(include=[np.number])` for numerics and `include=["object", "category", "bool"]` for categoricals.
2. For each non-numeric column, attempts `pd.to_datetime(sample, errors="coerce")` on the first 25 non-null values; declares it date-like if $\geq 70\%$ parse successfully.
3. Computes missing counts + percentages, duplicate count, memory in MB.
4. Numeric summary: pandas `describe()` – count, mean, std, min, 25%, 50%, 75%, max – rounded to 3 decimals.
5. Categorical summary: top-10 counts per column.
6. Pearson correlation matrix on all numeric columns.
7. Stores a 15-row preview for the dashboard's table view.

Output schema

shape, columns, dtypes, numeric_columns, categorical_columns, date_columns, missing_counts, missing_percent, duplicate_rows, duplicate_percent, memory_usage_mb, numeric_summary, categorical_summary, correlations, preview.

On showcase_v3_full.csv: shape=(10110, 32); ≈ 15 numeric, ≈ 13 categorical columns; application_date and default_notice_date flagged as dates; duplicate_rows=110; memory ≈ 3 MB.

3.2 column_roles.py [multi-label semantic roles]

Assigns a *set* of semantic roles to every column. Pure pandas / regex; no ML.

Libraries & key functions

pandas, numpy, re.

Algorithm

For each column, role assignment is driven by:

- **Dtype** $\rightarrow$ numeric, boolean, categorical.
- **Cardinality.** constant (≤ 1 unique), binary ($= 2$), low_cardinality (2–10), high_cardinality (unique_ratio > 0.80), unique_like (> 0.95).
- **Name keyword banks.** ID (id, uuid, account, ...), time (date, timestamp, year, ...), price (price, amount, fee, ...), volume (volume, qty, count, ...), sensitive (age, gender, race, ...), sequence, target hint.
- **Content date check.** `pd.to_datetime` on a 30-row sample; $\geq 70\%$ parse $\rightarrow$ time_like.
- **Text length.** avg length > 50 or max $> 200 \rightarrow$ long_text; otherwise short_text.
- **Missingness bucket.** severe_missing ($\geq 90\%$), very_high_missing, high_missing, moderate_missing, low_missing, complete.
- **Targets.** binary or low_cardinality non-id columns become target_candidate; numeric with > 10 unique values become regression_target_candidate.

On showcase_v3_full.csv: account_id $\rightarrow$ id_like; application_date + default_notice_date $\rightarrow$ time_like; age, gender, region, income $\rightarrow$ sensitive; loan_amount, monthly_payment, total_payment $\rightarrow$ price_like; customer_feedback $\rightarrow$ long_text; defaulted, is_autopay, has_collateral $\rightarrow$ binary + target_candidate.

3.3 dataset_signals.py [boolean + numeric flags]

Libraries & key functions

pandas, numpy, re (email pattern); imports column_roles.

Algorithm

1. Email detection: regex `\textasciicircum [\textasciicircum @\textbackslash s]+\textasciicircum @\textbackslash s]+\textbackslash .[\textasciicircum @\textbackslash s]+\$` on a 100-row sample; column flagged if $\geq 50\%$ match.
2. Long-text detection: avg length > 50 or max > 200 on the non-null entries.
3. Leakage heuristic: any non-id numeric pair with absolute Pearson correlation ≥ 0.98 raises `has_potential_leakage`.
4. Bid-ask special case: both `bid` and `ask` columns present (case-insensitive).
5. Returns 30+ flags plus the rich `column_roles` object.

On showcase_v3_full.csv: `has_potential_leakage=True` (loan / payment trio is at $\rho \approx 1.0$); every other expected flag also true.

3.4 analysis_inventory.py [declarative registry]

A single Python list of analysis descriptors. Each entry has `id`, `name`, `category`, `modes` (allow-list of {quick, standard, deep, research}), `priority`, `requires` (map of signal $\rightarrow$ expected value), `requires_any` (list of alternative requirement groups), optional `min_rows` and `requires_user_target`, plus outputs. There are 28 entries.

3.5 analysis_selector.py [signal-driven router]

Algorithm

For every entry in the inventory:

- Skip if the current analysis mode is not in `modes`.
- Skip if any `requires` signal is unmet, or if `min_rows` is not satisfied.
- Skip if `requires_any` groups all fail.
- Recommend (not select) if the analysis needs a target and none is set.
- Otherwise select.

Output: `selected_analyses`, `skipped_analyses`, `recommended_analyses`, plus a counts summary.

On showcase_v3_full.csv: in deep mode with `target=default`, all 28 analyses *selected*, 0 skipped, 0 recommended-after-target.

3.6 advanced_indicators.py [utility / fairness / freshness]

Libraries & key functions

pandas, numpy.

Algorithm

Column utility. Score $\in [0, 100]$:

$\text{score} = 100 - \min(40, \text{missing}\%) - 40 \cdot \mathbb{I}[\text{constant}] - 30 \cdot \mathbb{I}[\text{unique_ratio} > 0.9 \ \& \ \text{non-numeric}]$.

Buckets: ≥ 80 High, ≥ 50 Moderate, else Low.

Anomaly count. For every numeric column, IQR fence test; per-row anomaly score is the fraction of numeric columns flagging it. Top-10 rows reported.

Fairness review. Returns columns whose name matches the sensitive keyword bank (gender, sex, age, race, religion, region, location, income, salary, caste).

Freshness. Picks the first column with $\geq 70\%$ date-parse rate; reports oldest and latest record.

On showcase_v3_full.csv: utility ranks customer_feedback, credit_score, loan_amount high; account_id low; freshness window 2022-01-01 $\rightarrow$ 2024-12-31; fairness review marks age, gender, region, monthly_income, annual_income.

3.7 signal_engine.py [display cards]

Pure presentation logic that turns the underlying flags into seven human-readable cards: Data Completeness, Duplicate Records, Outlier Detection, Data Consistency, ML Readiness, Anomaly Detection, Fairness Review, plus a Temporal Data card when a date column exists. Each card has name, icon, status, severity, evidence, recommendation.

4 Quality Scoring

4.1 health_score.py [0–100 quality grade]

Libraries & key functions

pandas, numpy.

Math / formulas

Component scores (each clipped to $[0, 100]$):

$$\begin{aligned}\text{completeness} &= 100 - \text{missing}\%, \\ \text{uniqueness} &= 100 - \text{duplicate}\%, \\ \text{outlier safety} &= 100 - \text{IQR-outlier}\%, \\ \text{consistency} &= 100 - (\text{constant}\% + \text{high-card}\%).\end{aligned}$$

Final score:

$$\text{Health} = 0.30 \cdot \text{completeness} + 0.20 \cdot \text{consistency} + 0.20 \cdot \text{uniqueness} + 0.20 \cdot \text{outlier safety} + 10.$$

Labels: ≥ 90 Excellent, ≥ 75 Good, ≥ 60 Moderate, ≥ 40 Poor, else Critical.

On showcase_v3_full.csv: Good label, score in the mid-70s. Missing $\approx 3.5\%$, duplicate $\approx 1\%$, outlier $\approx 2\%$, no constants.

4.2 ml_readiness.py [ready-for-modelling score]

Math / formulas

$$\text{score} = 100 - \min(30, \text{missing}\%) - \min(15, \text{dup}\%) - \min(20, 2 \cdot |\text{cat cols}|).$$

Labels: ≥ 85 Ready, ≥ 70 Mostly Ready, ≥ 50 Partially, else Not Ready. Recommendations are static – a checklist of "handle missing", "encode categoricals", "remove duplicates", "select target".

On `showcase_v3_full.csv`: ~70–80 (Mostly Ready). The encoding penalty caps at 20 because there are ≥ 10 categoricals.

5 Statistics

5.1 deep_statistics.py [baseline stats, legacy v1]

Libraries & key functions

pandas, numpy, scipy.stats (normaltest, chi2_contingency, zscore).

Algorithm

1. Per numeric column: count, mean, median, std, min, max, Q_1 , Q_3 , IQR, skew, kurtosis (with descriptive labels), classical z -outlier count ($|z| > 3$), normality via D'Agostino-Pearson (`normaltest`) on samples up to 5000 rows.
2. Correlation insights: full numeric Pearson matrix, ranked top-10 absolute pairs, plus a “high-correlation” subset with $|\rho| \geq 0.8$.
3. Per categorical column: top-10 value counts + percent, plus a cardinality bucket (Constant, Low, Medium, High).
4. Chi-square tests on every categorical pair where each side has ≥ 2 unique values.

5.2 deep_statistics_v2.py [research-grade battery]

Libraries & key functions

pandas, numpy, scipy.stats: shapiro, normaltest, anderson, bootstrap, pearsonr, spearmanr, kendalltau, pointbiseiralr, mannwhitneyu, kruskal, chi2_contingency, plus distributions norm, lognorm, expon, gamma, weibull_min, beta.

Algorithm

Per numeric column:

1. Descriptives: count, mean, median, std, min, max, Q_1 , Q_3 , IQR, skew, kurtosis with labels.
2. **Three outlier views:** IQR ($1.5 \cdot \text{IQR}$ fences), classical z -score ($|z| > 3$), robust MAD- z ($|z^*| > 3.5$).
3. **Normality.** Shapiro–Wilk for $n \leq 5000$; D'Agostino–Pearson for $n \geq 20$ (samples 5000 if larger); Anderson–Darling statistic vs the 5% critical value. Verdict prefers Shapiro p -value when available.
4. **Distribution fit.** Fits each candidate by MLE, scores by AIC, picks the lowest. Beta is fitted on a $(0, 1)$ -rescaled copy when needed; positive-support distributions skip columns with non-positive values.
5. **Bootstrap CIs.** 999 BCa resamples for the 95% CI on the mean and on the median. Falls back to percentile method on degenerate data.

Per categorical column:

- Top-10 value counts + Shannon entropy and entropy ratio.

Bivariate (capped at `max_pairs=20` per type to bound runtime):

- Numeric pairs: Pearson, Spearman, Kendall (all with p -values).
- Categorical pairs: χ^2 + Cramer's V (contingency tables capped at 30 categories per side).
- Binary categorical vs numeric: point-biserial (encodes the binary as $\{0, 1\}$).
- Group differences: Mann–Whitney U for 2 groups, Kruskal–Wallis H for 3–12 groups.

All numbers are rounded to 6 decimals via a JSON-safe coerce that drops NaN and Inf.

On showcase_v3_full.csv: strong correlations $\text{loan_amount} \leftrightarrow \text{monthly_payment} \leftrightarrow \text{total_payment} \approx 1.0$; $\text{annual_income} \leftrightarrow \text{monthly_income} \approx 1.0$; loan_amount highly skewed, best-fit log-normal; $\text{credit_score} \leftrightarrow \text{defaulted}$ negative point-biserial; Mann–Whitney significant for employment_type vs defaulted ; weak Cramer’s V for region vs product_tier .

6 Anomaly Detection

6.1 anomaly_ensemble.py [6+ detectors, mean ensemble]

Libraries & key functions

numpy, pandas, sklearn.ensemble.IsolationForest, sklearn.neighbors.LocalOutlierFactor, sklearn.covariance.EllipticEnvelope, sklearn.covariance.MinCovDet, sklearn.impute.SimpleImputer, sklearn.preprocessing.StandardScaler. Optional: pyod.models.ecod.ECOD, pyod.models.copod.COPOD.

Algorithm

1. **Prepare matrix.** Drop constant numerics, cap to 30 columns (keep the most populated), median-impute missing, then `StandardScaler`.
2. Run each detector and produce a $[0, 1]$ score:
 - *Isolation Forest* – 200 trees, contamination configurable (default 5%), $n_{\text{jobs}} = -1$. Score is $-\text{score_samples}$ min-max scaled.
 - *Local Outlier Factor* – $n_{\text{neighbors}} = \min(20, \max(5, n/20))$. Score is $-\text{negative_outlier_factor_}$ min-max scaled.
 - *Elliptic Envelope* – robust Gaussian fit.
 - *MAD robust z* – per-row mean of $|x - \tilde{x}| / (1.4826 \cdot \text{MAD})$ across columns.
 - *Mahalanobis* – distance under `MinCovDet` robust covariance, fallback to plain covariance with Σ^{-1} regularised by $10^{-6} I$.
 - *ECOD / COPOD* – when `pyod` is installed.
3. **Ensemble.** Per-row mean across all available detectors.
4. **Flagging.** Rows with ensemble score $\geq \text{threshold}$ (default 0.6).

Output schema

available, n_rows_scored, used_columns, detectors_run, threshold, contamination, flagged_count, ensemble_summary (min/mean/median/max/p95/p99), top- k rows with each detector score plus the ensemble.

On showcase_v3_full.csv: the 45 seeded outliers score highest (loan / payment trio scaled by $\{0.04, 14, 22\}$). `flagged_count` ≈ 40 –80; `ensemble_summary.p99` ≈ 0.85 .

6.2 advanced_indicators.py::calculate_anomalies [lightweight IQR pass]

A coarse, single-pass scorer. Per row, $\text{score} = (\text{number of numeric columns flagging it under IQR fences}) / (\text{numeric column count})$. Returns `anomalous_rows` count, `highest_score`, and the top-10 rows above 0. Used by the AI report context and the static visualizer.

7 Relationships

7.1 multicollinearity.py [VIF + redundancy groups]

Libraries & key functions

sklearn.linear_model.LinearRegression, sklearn.impute.SimpleImputer, pandas, numpy.

Algorithm

1. Drop the target column (if any) and constant numerics; cap to 30 columns by non-missing count; median-impute the remainder.
2. For each feature j , regress it on all others with `LinearRegression`; collect R_j^2 .
3. Compute $VIF_j = 1/(1 - R_j^2)$, capped at 999 when $R_j^2 \geq 0.999$.
4. Severity: ≥ 10 High, ≥ 5 Medium, else Low.
5. Redundant groups: pairs with $|\rho| \geq 0.95$ are merged into groups; report each group with a recommended action.

On showcase_v3_full.csv: severe High cluster – `loan_amount`, `monthly_payment`, `total_payment` all $VIF \sim 999$. Second cluster: `annual_income`, `monthly_income`. `status = "High multicollinearity detected"`.

7.2 target_leakage.py [target-aware feature audit]

Algorithm

For every non-target column:

1. *Same-value ratio*. Where both feature and target are non-null and overlap ≥ 10 rows, fraction of equal-string cells.
2. *Numeric correlation*. Pearson when both sides are numeric.
3. *Risk classification*. same-ratio $\geq 0.98 \rightarrow$ **Critical**; $|\rho| \geq 0.98 \rightarrow$ **High**; $|\rho| \geq 0.9$ or feature/target name overlap $\rightarrow$ **Medium**; else Low.

Sorted output prefers Critical first.

On showcase_v3_full.csv: `default_notice_date` flagged **Critical** – it is filled only when `defaulted=1`. The strong negative correlation between `credit_score` and `defaulted` stays Low because $|\rho|$ is below 0.9.

7.3 target_analyzer.py [target shape + warnings]

Algorithm

1. Auto-detect task type. Numeric with ≤ 10 unique values $\rightarrow$ classification; numeric with `unique_ratio` $> 0.05 \rightarrow$ regression; otherwise classification.
2. Classification details: per-class count + percent, majority ratio, minority count, imbalance label (**Severely Imbalanced** $\geq 90\%$, **Imbalanced** $\geq 75\%$, etc.).
3. Regression details: count, mean, median, std, min, max, skewness + label, IQR-outlier count.
4. Warnings: missing target, imbalance, minority < 10 , highly skewed regression target, regression outliers.

On showcase_v3_full.csv: `task_type=classification`; 2 classes; majority ratio $\approx 91\%$; `imbalance_status="Imbalanced"`; warning emitted.

8 ML Lab

8.1 ml_preprocessing.py [shared preprocessor]

Libraries & key functions

```
sklearn.compose.ColumnTransformer,          sklearn.pipeline.Pipeline,
sklearn.preprocessing.OneHotEncoder,       sklearn.preprocessing.StandardScaler,
sklearn.preprocessing.FunctionTransformer, sklearn.impute.SimpleImputer.
```

Algorithm

Why a single preprocessor? Leaderboard, explainability, diagnostics, and feature-importance modules all call `prepare_ml_matrix` so the dropped-column rules are consistent.

1. Drop rows with missing target.
2. Drop columns that are: empty (non-missing < 5), constant, ID-like by name (`id`, `uuid`, `account`, `customer\id`, ...), time-like by name when the dtype is non-numeric, non-numeric with unique-ratio > 0.95 .
3. Numeric pipeline: `SimpleImputer(strategy="median")` $\rightarrow$ `StandardScaler`.
4. Categorical pipeline: `SimpleImputer(strategy="constant", fill_value="__missing__")` $\rightarrow$ `stringify` (cast every cell to `str` so mixed bool/str/sentinel columns sort) $\rightarrow$ `OneHotEncoder(handle_unknown="ignore", sparse_output=False)`.
5. Returns `X`, `y`, lists of numeric / categorical features, the dropped-columns audit log, warnings, and a usable flag (need ≥ 1 feature and ≥ 20 rows).

On `showcase_v3_full.csv`: drops `account_id`, `application_date`, `default_notice_date`; keeps about 24 features.

8.2 model_leaderboard.py [CV scoreboard]

Libraries & key functions

```
sklearn.linear_model.LogisticRegression,      Ridge,          Lasso;
sklearn.neighbors.KNeighborsClassifier        /      KNeighborsRegressor;
sklearn.ensemble.RandomForestClassifier      /      RandomForestRegressor;
sklearn.ensemble.GradientBoostingClassifier / GradientBoostingRegressor;
sklearn.dummy.DummyClassifier / DummyRegressor; sklearn.model_selection.StratifiedKFold
/ KFold / cross_validate / train_test_split; sklearn.metrics: accuracy_score,
precision_score, recall_score, f1_score, r2_score, mean_absolute_error,
mean_squared_error, brier_score_loss. Optional: xgboost.XGBClassifier /
XGBRegressor, lightgbm.LGBMClassifier / LGBMRegressor.
```

Algorithm

1. Build the registry. **Classification:** `Dummy(most_frequent)`, `LogReg(class_weight="balanced", max_iter=2000)`, `KNN( $k = 7$ )`, `RF(200 trees, depth 10, class_weight="balanced")`, `GB(150 trees, depth 4)`, plus `XGBoost` and `LightGBM` when importable. **Regression:** `DummyRegressor(mean)`, `Ridge( $\alpha = 1$ )`, `Lasso( $\alpha = 0.01$ )`, `KNN( $k = 7$ )`, `RF`, `GB`, plus `XGB` / `LGBM`.
2. Coerce target labels. Non-numeric classification targets are encoded with `astype("category").cat.codes` (`XGBoost` requires integer labels).
3. CV. `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` for classifi-

cation (capped by smallest class count); KFold for regression. Run `cross_validate` with the metrics map and `n_jobs=-1`.

4. For each model, report mean + std for every metric, plus fit time. Sort by primary score (`f1_weighted` for classification, `r2` for regression). The non-Dummy winner gets a 75/25 holdout via `train_test_split` (stratified for classification when feasible).
5. Holdout extras for classification include the Brier score for binary targets when `predict_proba` is available.

Output schema

available, task_type, primary_metric, n_rows, n_features, leaderboard (per row: model, all metric means + stds, fit_time_s, primary_score), winner, holdout block.

On showcase_v3_full.csv: winner is RandomForest, XGBoost, or LightGBM. `f1_weighted` in the 0.91–0.95 range thanks to the imbalanced target. Brier score ~0.05–0.08. Dummy F1 ≈ 0.83 (always predicts the majority class).

8.3 baseline_model.py [model vs dummy]

Libraries & key functions

```
sklearn.ensemble.RandomForestClassifier / RandomForestRegressor;
sklearn.dummy.DummyClassifier / DummyRegressor; sklearn.compose.ColumnTransformer;
sklearn.pipeline.Pipeline; sklearn.metrics: accuracy / precision / recall / f1
weighted, R2, MAE, MSE.
```

Algorithm

A simpler legacy sanity check.

1. Re-run the same drop rules as `ml_preprocessing` (column keeps are independent of the v2 preprocessor for backward compatibility).
2. Train RF (`n_estimators=120`, `max_depth=8`, `class_weight="balanced"` for classification) and a Dummy on a 75/25 split.
3. Report metrics for both side by side, primary metric (`f1_weighted` / `r2`), train + test sizes, and the list of dropped columns.

On showcase_v3_full.csv: RF beats the Dummy by roughly +0.08 `f1_weighted`, confirming the signal is real.

8.4 explainability.py [SHAP on the winner]

Libraries & key functions

```
shap.TreeExplainer, shap.LinearExplainer, shap.summary_plot;
sklearn.inspection.permutation_importance; sklearn.pipeline.Pipeline;
sklearn.model_selection.train_test_split; matplotlib.pyplot (figure saved
as PNG).
```

Algorithm

1. Rebuild the winning estimator from the leaderboard.
2. Re-prepare the matrix via `prepare_ml_matrix`; classification targets are integer-encoded.
3. 75/25 `train_test_split` (stratified when feasible). Fit the full pipeline on train.

4. Sample up to `sample_size=200` rows from the test slice for SHAP. Transform through the fitted preprocessor.
5. *Tree models*: `shap.TreeExplainer`. *Linear models*: `shap.LinearExplainer` with a 100-row background sample (transformed). Otherwise fall back to permutation importance.
6. Multiclass / 3-D SHAP outputs are collapsed: global = mean across rows of `|SHAP|`; per-row plot uses positive class for binary, mean across classes otherwise.
7. Per-row stories: top 3 rows by total `|SHAP|`, each with its 5 strongest contributions.
8. Save the SHAP beeswarm to `static/charts/<dataset_id>_shap_summary.png`.
9. Always run permutation importance on the test slice as a cross-check (5 repeats, `random_state=42, n_jobs=-1`).

On `showcase_v3_full.csv`: top global features {`credit_score`, `debt_to_income`, `num_late_payments`, `employment_type`, `has_collateral`, `is_autopay`, `interest_rate`, `loan_amount`}; saved beeswarm chart.

8.5 `model_diagnostics.py` [secondary RF audit]

Algorithm

Independent RF run on the same matrix; uses `prepare_xy` (the legacy drop rules).

- Classification: `classification_report` (precision / recall / f1 per class), `confusion_matrix`, 5-fold CV `f1_weighted`.
- Regression: residual summary, top-10 worst predictions (largest absolute error), 5-fold CV R^2 .
- Warnings: CV mean $> 0.98 \rightarrow$ "check target leakage"; zero residual std $\rightarrow$ "suspicious".

On `showcase_v3_full.csv`: confusion matrix dominated by the negative class; the leakage warning is *not* fired here because `default_notice_date` is already dropped as time-like.

8.6 `feature_importance.py` [impurity + mutual info]

Libraries & key functions

`sklearn.ensemble.RandomForestClassifier` / `RandomForestRegressor`,
`sklearn.feature_selection.mutual_info_classif` / `mutual_info_regression`.

Algorithm

1. Fit RF (`n_estimators=120, max_depth=8`) on the preprocessed matrix.
2. Read `feature_importances_` (Gini impurity-based importance for classifiers; variance reduction for regressors).
3. Collapse one-hot dummies back to the parent column by *summing* the importances of the dummies.
4. Mutual information: median-impute numerics, label-encode categoricals, run `mutual_info_classif` or `mutual_info_regression` (`random_state=42`).

On `showcase_v3_full.csv`: headline features mirror SHAP, with mutual information pushing `nps_score` and `num_inquiries_90d` a few slots higher.

8.7 segment_analysis.py [group-wise behaviour]

Algorithm

1. Pick up to 5 categorical columns with 2–12 unique values.
2. If the target is numeric: `groupby(segment)` on the target and report count, mean, median, std, min, max. Otherwise report group share of total rows.
3. Returns the column list, the per-segment groups, and a textual interpretation.

On `showcase_v3_full.csv`: segments `region`, `employment_type`, `product_tier`, `channel`, `education`. Switches to numeric-target mode if you re-run with `interest_rate`.

9 Time, Text, Drift

9.1 time_series.py [auto-detect → STL → forecast]

Libraries & key functions

pandas, numpy; statsmodels.tsa.stattools.adfuller / kpss,
statsmodels.tsa.stattools.acf / pacf, statsmodels.tsa.seasonal.STL,
statsmodels.tsa.holtwinters.ExponentialSmoothing.

Algorithm

1. **Date detection.** For each column, compute parse rate via `pd.to_datetime(..., errors="coerce", format="mixed")`, and a monotone score = $\max(\text{fraction increasing, fraction decreasing})$. Reject low-cardinality numeric columns whose name is not date-like (so binary integers don't get parsed as nanosecond epochs). Score = $0.6 \cdot \text{parse rate} + 0.3 \cdot \text{monotone} + 0.1 \cdot \mathbb{I}[\text{name is date-like}]$. Highest score wins.
2. **Frequency inference.** Compute the median inter-arrival time in seconds and bucket into {second, minute, hour, day, week, month, quarter, year, irregular}. Also try `pd.infer_freq(parsed.head(200))`.
3. **Numeric series picker.** If a target is given and numeric, use it. Otherwise pick the highest-variance non-ID numeric column.
4. **Stationarity.** ADF (`autolag="AIC"`) and KPSS (`regression="c", nlags="auto"`). Combine to one of {stationary, non-stationary, trend-stationary, difference-stationary, inconclusive}.
5. **Period via FFT.** Compute the real-FFT magnitude spectrum, ignore DC, pick the dominant peak, $\text{period} = \text{round}(n/\text{peak})$. Sanity-bound by frequency label.
6. **STL decomposition.** `STL(series, period=period, robust=True).fit()`; report trend / seasonal / residual previews and Hyndman strength scores F_T, F_S .
7. **Autocorrelation.** ACF + PACF up to $\min(40, n/5)$ via `acf(..., fft=True)` and `pacf`.
8. **Forecast.** Holt–Winters additive trend, additive seasonality if $\text{period} \geq 2$ and $n \geq 2 \cdot \text{period}$. Train on $n - h$ rows where $h = \min(\max(\text{period}, 12), n/4)$. Naive baseline = last value. Report MAE + RMSE for both.

On showcase_v3_full.csv: detected column `application_date`; frequency label `day` (median 1 day inter-arrival); period ~ 7 (weekly); paired numeric `loan_amount` (highest variance non-ID column); ADF + KPSS verdict *trend-stationary*; HW beats naive on MAE.

9.2 text_profile.py [TF-IDF + LSA preview]

Libraries & key functions

`re`, `collections.Counter`, `numpy`, `pandas`; `nltk.corpus.stopwords`
when available (falls back to a built-in list of ~ 80 stop-
words); `sklearn.feature_extraction.text.TfidfVectorizer`,
`sklearn.decomposition.TruncatedSVD`.

Algorithm

Detection. A column is text-like when: object/string dtype and $\geq 30\%$ of values have ≥ 2 tokens and average length ≥ 8 chars and (high cardinality but avg length ≥ 20 chars or any cardinality with avg length ≥ 8 chars).

For up to 5 detected columns:

1. Length stats via `str.len()`: average, min, max, median.
2. Tokenisation: `re.findall(r"\textbackslash b[\textbackslash w'-]+\textbackslash b", lower)`. Stopwords filtered when computing top terms.
3. Vocabulary stats: total tokens, vocab size, lexical diversity = $|V|/N$.
4. Avg sentence length: split on `[.!?]` and count tokens per sentence.
5. Top 15 unigrams + top 12 bigrams via `Counter`.
6. Pattern hits via regex: emails, URLs, phone-like numbers, mentions, hashtags, monetary tokens. Reports row-counts + examples.
7. Language guess (heuristic): ASCII fraction + English-stopword overlap classify the column as English / Latin-non-English / non-Latin / unknown.
8. Sentiment-lite: ratio of words from a small positive seed list (`{great, love, helpful, smooth, fast, useful, ...}`) and a negative one (`{terrible, broken, slow, useless, disappointed, ...}`); polarity = positive–negative ratio.
9. LSA: `TfidfVectorizer(max_features=1024, stop_words="english", ngram_range=(1,2))` then `TruncatedSVD(n_components=2, random_state=42)`. Returns 200 sample points (x, y) plus the explained variance ratios.

On showcase_v3_full.csv: customer_feedback profiled. Top unigrams skew towards positive seeds for non-defaulters and negative seeds for defaulters; LSA scatter shows two soft clusters.

9.3 drift_analysis.py [compare two dataframes]

Libraries & key functions

numpy, pandas, scipy.stats.ks_2samp, scipy.stats.chi2_contingency.

Algorithm

Numeric column.

1. Coerce to float, drop NaN/Inf.
2. PSI on 10 quantile bins from the reference.
3. Two-sample Kolmogorov–Smirnov (`ks_2samp`); flag at $p < 0.01$.
4. Mean shift z in pooled SD units.
5. 20-bin histogram preview shared across ref + cur.

Categorical column.

1. PSI on category proportions (Laplace smoothed by 10^{-6} to avoid log 0).
2. Chi-square independence on a $2 \times K$ contingency table (top 30 categories).
3. Lists new and missing categories.

PSI severity: < 0.10 stable, < 0.25 minor, < 0.50 moderate, ≥ 0.50 severe. KS upgrades a stable PSI to minor when significant.

`split_by_date`. Sorts by parsed date, cuts at `ratio` (default 0.5), returns (older, newer). Used by `/api/compare-self`.

On showcase_v3_full.csv: on showcase_v3_full self-compare with `ratio=0.5`, verdict is *stable* (no concept drift). The v4 build's mid-2023 drift produces *severe* PSI on `defaulted` and `interest\rate`.

10 Cleaning & Visuals

10.1 cleaner.py [cleaning pass + before/after]

Libraries & key functions

pandas, internal health_score, profiler, security.sanitize_csv_value.

Algorithm

1. Drop exact duplicate rows.
2. For each column with missing values: numeric → fill with median; non-numeric → fill with mode (or "Unknown" when no mode exists).
3. Compute health score before and after (calls build_profile and calculate_health_score twice).
4. Pass each cell through sanitize_csv_value to prevent CSV injection; write cleaned/<dataset_id>_cleaned.csv.
5. Return: list of actions, before/after health, output path, before/after missing + duplicate counts.

On showcase_v3_full.csv: removes 110 duplicates; imputes ~ 3.5% missing across age, nps_score, customer_feedback, existing_debt, num_inquiries_90d; health score climbs ~ 5–10 points.

10.2 chart_planner.py [decides which charts to render]

Pure planning logic – inspects column roles + signals + upstream phases and returns a list of chart specs. Each spec is a dict with id, type, title, reason plus type-specific keys (column, columns, feature, target). Chart types include: health components, schema composition donut, missing-by-column, cleaning risk, cardinality strip, numeric distribution + box + violin, six-up numeric overview, categorical count + Pareto, correlation heatmap, top correlations, column utility, cleaning impact, anomaly scores, target distribution, target-vs-feature scatter, leaderboard bars, feature-importance bars, time-series trend, bivariate highlights, bid-ask spread.

On showcase_v3_full.csv: ~ 18–22 charts produced for the showcase.

10.3 visualizer.py [renders the chart specs]

Libraries & key functions

matplotlib (Agg backend), seaborn, numpy, pandas; internal palette mirroring the dashboard tokens.

Algorithm

1. Apply the editorial-dark palette as plt.rcParams defaults (cream ink on near-black).
2. For every chart spec, dispatch to a renderer (e.g. _chart_health, _chart_distribution, _chart_correlation, _chart_target_distribution, _chart_target_vs_feature, _chart_leaderboard, _chart_feature_importance).
3. Common layout helpers add a left-aligned title and a monospaced eyebrow / subtitle, hide top + right spines, draw thin grids only on the data axis.
4. Save PNG to static/charts/<dataset_id>_<id>.png at 200 DPI with bbox_inches="tight" and the page-background face colour.

On showcase_v3_full.csv: ~ 20 PNGs land on disk; total ~ 3–5 MB.

11 AI & RAG Layer

11.1 ai_insights.py [report writer]

Libraries & key functions

json, urllib.request for OpenRouter; optional ollama client.

Algorithm

1. Compact context: profile (without preview), health, signals, ML readiness, advanced indicators, role summary, dataset signals.
2. Build the structured prompt with section headings: Executive Summary → Data Quality Risks → Key Insights → Cleaning Recommendations → ML Readiness Assessment → Warnings → Next Steps.
3. Call OpenRouter first (default model meta-llama/llama-3.1-8b-instruct:free, temperature 0.2, max_tokens=1400). Fall back to Ollama (llama3.2) if OpenRouter fails. If both fail, run a deterministic Python fallback that builds the same Markdown sections from the computed stats.

On `showcase_v3_full.csv`: the report calls out the `default_notice_date` leakage, the loan-amount VIF cluster, the imbalanced target, and the sensitive columns flagged by the fairness review.

11.2 agent_brief.py [compact dataset packing]

Algorithm

Builds a ≤ 9 KB JSON brief covering: identity (id, filename, mode, shape, target), schema (top 30 columns with dtype, role list, missing %, top categorical values), quality (health components, ML readiness), signals (top 6), ML headlines (target analysis, leaderboard winner + top-3, SHAP top-8, multicollinearity top-6, target-leakage suspects, diagnostics), time / text / drift snippets, top-6 correlations, ~5 sample rows, and the AI narrative truncated to 1200 chars. Progressive trimming keeps total bytes under the cap.

11.3 agent_tools.py [tool registry]

Exposes nine audited tools the agent can call: `get_section(path)`, `list_columns()`, `column_summary(name)`, `run_query(expression)` (sandboxed), `get_top_anomalies(k)`, `get_leaderboard()`, `get_explainability_t`, `search_facts(question, k)`, `get_dataset_brief(...)`. Every tool result is JSON-safe and truncated when it would exceed 6 KB.

11.4 ai_agent.py [planner / executor / critic / writer]

Libraries & key functions

pydantic (BaseModel for Plan / ToolCall / Verdict), json, urllib.request for OpenRouter, ollama.Client for local models.

Algorithm

Fast path (default). Build the dataset brief, run RAG retrieve top- k , then a single writer LLM call. One LLM hop; sub-second on hosted models.

Full path.

1. Planner LLM call in JSON mode – chooses 1–3 tools.
2. Executor runs each tool via `run_tool`.

3. Critic LLM call in JSON mode – verdict object with **accept**, **reason**, optional **repair** list.
4. If repair is requested (max 1 iteration), re-execute and re-write.
5. Writer LLM call composes the final Markdown answer in the Answer / Evidence / Recommendation / Limitation format.

Routing policy via LLM_PREFER: auto (Ollama then OpenRouter then heuristic), ollama, openrouter, or off (forces the heuristic writer that quotes RAG facts directly). Cloud Ollama models (deepseek-v3.1:671b-cloud, gpt-oss:120b-cloud, kimi-k2.5:cloud) are tried first when DATALENS_OLLAMA_ALLOW_CLOUD=1.

11.5 rag_index.py [fact index + retrieval]

Libraries & key functions

numpy; ollama (nomic-embed-text embeddings) when reachable; sklearn.feature_extraction.text.TfidfVectorizer fallback. concurrent.futures.ThreadPoolExecutor parallelises embedding requests (default 8 workers).

Algorithm

1. Walk the analysis dict and emit one fact per leaf, formatted as <dotted path>: <summary>. Excluded paths: chart filenames, AI report text, raw correlation matrix, full preview, per-row anomaly detail, per-column deep-stats trees.
2. Embed corpus + query with nomic-embed-text concurrently. On failure, fall back to TF-IDF (1–2 grams, 4096 max features, token regex `\textbackslash b[\textbackslash w-]{2,}\textbackslash b`).
3. Cosine similarity top-*k* via either `argsort` (small *k*) or `argpartition` (large).
4. Render as a bulleted Markdown block, capped at 4000 chars.

On **showcase_v3_full.csv**: ~ 700–900 facts indexed for the showcase.

11.6 safe_pandas.py [AST-allowlist sandbox]

Libraries & key functions

ast, numpy, pandas.

Algorithm

Parses the user expression with `ast.parse(expr, mode="eval")` and walks the tree with two limits: max depth 30, max length 600 chars. Allow-lists:

- Names: {df, np, pd, len, abs, min, max, round, sum, sorted}.
- Attributes: ~80 safe pandas / numpy attributes (loc, iloc, head, tail, groupby, agg, mean, sum, select_dtypes, drop_duplicates, str, dt, ...).
- Operators: arithmetic, comparison, boolean, indexing.

Refused: imports, lambdas, dunder, IO (`open`), `exec`, `eval`, `compile`, `globals`, `locals`. Any disallowed name / attribute / node type raises `UnsafeExpression`. Compiled AST is evaluated under `__builtins__`: `\{\}` so even if something slips, no built-in is reachable. Output is JSON-coerced and truncated to 50 rows / 6 KB.

On **showcase_v3_full.csv**: `run_query` can execute `df["loan_amount"].describe()` or `df.groupby("region")["defaulted"].mean()` but rejects anything risky.

12 Reports

12.1 report_generator.py [thin wrapper]

A one-liner that forwards to `pdf_report_builder.generate_pdf_report` and writes the PDF to `reports/`.

12.2 pdf_report_builder.py [editorial-dark PDF]

Libraries & key functions

`matplotlib` (Agg backend), `matplotlib.backends.backend_pdf.PdfPages`, `matplotlib.patches.FancyBboxPatch` / `Rectangle`, `datetime`, `textwrap`, `unicodedata`.

Algorithm

A custom composer ("Composer" class) lays out one Matplotlib figure per A4 page using inches as coordinates. Pages flow as:

- Cover. Hero band, big title, dataset name, KPI tiles (Health Score, ML Readiness, Charts, Signals).
- Contents. Numbered section list with blurbs.
- Sections. Executive Summary, Quality, Signals, Cleaning, Deep Stats, ML, Time, Text, AI, Charts. Each section is a divider page with a coloured eyebrow and big title, followed by KPI tiles, callouts, bullets, key/value tables.
- Visual Evidence. Chart images embedded two per page with headline + caption.

On `showcase_v3_full.csv`: produces a ~ 15–25 page PDF with every chart embedded.

13 Library inventory and dependency map

Every external library used anywhere in `modules/`, with the modules that import it. All ML / stats are pinned to `scikit-learn`, `statsmodels`, `scipy`, `shap`; everything else is glue.

Library	Used in / what for
<code>pandas</code>	Universal – I/O, slicing, profile, correlations, group-bys, time handling.
<code>numpy</code>	Universal – vectorised math, FFT, histograms, masked arrays.
<code>scipy.stats</code>	<code>deep_statistics(_v2)</code> , <code>drift_analysis</code> , <code>text_profile</code> (Shapiro, <code>normaltest</code> , <code>anderson</code> , <code>bootstrap</code> , <code>ks_2samp</code> , <code>chi2_contingency</code> , <code>mannwhitneyu</code> , <code>kruskal</code> , <code>pearsonr</code> , <code>spearmanr</code> , <code>kendalltau</code> , <code>pointbiseriarr</code>).
<code>scikit-learn</code>	<code>ml_preprocessing</code> , <code>model_leaderboard</code> , <code>explainability</code> , <code>model_diagnostics</code> , <code>feature_importance</code> , <code>baseline_model</code> , <code>multicollinearity</code> , <code>anomaly_ensemble</code> , <code>rag_index</code> (TF-IDF), <code>text_profile</code> (TF-IDF + TruncatedSVD). Concrete classes: <code>LogisticRegression</code> , <code>Ridge</code> , <code>Lasso</code> , <code>KNN</code> , <code>RandomForest</code> , <code>GradientBoosting</code> , <code>Dummy</code> , <code>IsolationForest</code> , <code>LocalOutlierFactor</code> , <code>EllipticEnvelope</code> , <code>MinCovDet</code> , <code>Pipeline</code> , <code>ColumnTransformer</code> , <code>OneHotEncoder</code> , <code>StandardScaler</code> , <code>SimpleImputer</code> , <code>FunctionTransformer</code> , <code>LinearRegression</code> , <code>train_test_split</code> , <code>StratifiedKFold</code> , <code>KFold</code> , <code>cross_validate</code> , <code>permutation_importance</code> , <code>mutual_info_classif</code> / <code>mutual_info_regression</code> , <code>classification</code> & <code>regression</code> metric helpers, <code>TfidfVectorizer</code> , <code>TruncatedSVD</code> .
<code>statsmodels</code>	<code>time_series</code> (ADF, KPSS, ACF, PACF, STL, Holt–Winters).
<code>shap</code>	<code>explainability</code> (<code>TreeExplainer</code> , <code>LinearExplainer</code> , <code>summary_plot</code>).
<code>xgboost</code> (optional)	<code>model_leaderboard</code> (<code>XGBClassifier</code> / <code>XGBRegressor</code>).
<code>lightgbm</code> (optional)	<code>model_leaderboard</code> (<code>LGBMClassifier</code> / <code>LGBMRegressor</code>).
<code>pyod</code> (optional)	<code>anomaly_ensemble</code> (ECOD, COPOD).
<code>matplotlib</code>	<code>visualizer</code> , <code>pdf_report_builder</code> , <code>explainability</code> (SHAP plot).
<code>seaborn</code>	<code>visualizer</code> (heatmaps, hist + KDE, boxplots).
<code>nlTK</code> (optional)	<code>text_profile</code> (English stopwords).
<code>ollama</code>	<code>ai_agent</code> , <code>ai_insights</code> , <code>rag_index</code> (local model + embeddings).
<code>pydantic</code>	<code>ai_agent</code> (Plan / ToolCall / Verdict schemas).
<code>flask</code> , <code>werkzeug</code>	<code>app.py</code> + <code>security.py</code> (<code>secure_filename</code> , request handling).
<code>urllib.request</code>	<code>ai_insights</code> , <code>ai_agent</code> (OpenRouter HTTP client – avoids adding the <code>requests</code> dependency).
<code>tclobox</code> / <code>latex</code>	this report only.

14 Expected end-to-end run on showcase_v3_full.csv

Upload `showcase_v3_full.csv` with `target_column=defaulted` and `analysis_mode=deep`. Every column below corresponds to a dashboard tab.

Tab / module	Concrete output to expect
PROFILE	(10110, 32), 110 duplicates, full dtype + missing breakdown, 15-row preview.
ROLES	4 sensitive columns flagged, 1 id-like, 2 time-like, 3 price-like, 1 long-text, 4+ binary target candidates.

Tab / module	Concrete output to expect
HEALTH	Score 72–80 (Good).
ML READINESS	Score ~72–82 (Mostly Ready).
STATISTICS V1	Pearson top-pairs board led by the loan / payment trio at ~1.0; high-correlation list non-empty.
STATISTICS V2	Skewness + best-fit log-normal on <code>loan_amount</code> ; Shapiro $p < 10^{-3}$ on most numerics; significant Mann–Whitney for <code>employment\type</code> vs <code>defaulted</code> ; weak Cramer’s V across most categorical pairs; strong point-biserial for <code>credit\score</code> vs <code>defaulted</code> .
ANOMALIES (v2)	45+ rows flagged, ensemble $p99 \approx 0.85$, <code>detectors_run</code> ≥ 5 .
MULTICOLLINEARITY	Severe-High cluster at VIF ~999 across <code>loan_amount</code> , <code>monthly_payment</code> , <code>total_payment</code> ; second cluster <code>annual_income</code> , <code>monthly_income</code> .
TARGET LEAKAGE	Critical on <code>default_notice_date</code> ; everything else Low.
TARGET ANALYSIS	2 classes; majority $\approx 91\%$; Imbalanced.
LEADERBOARD	RF / XGB / LGBM tie around <code>f1_weighted</code> 0.91–0.95; Dummy at 0.83; Brier ~0.05–0.08.
SHAP	Beeswarm dominated by <code>credit_score</code> , <code>debt_to_income</code> , <code>num_late_payments</code> , <code>employment_type</code> , <code>has_collateral</code> ; saved PNG.
DIAGNOSTICS	Confusion matrix dominated by the negative class; CV F1 $\approx$ leaderboard score.
TIME SERIES	<code>application_date</code> , frequency <code>day</code> , period ~7, ADF + KPSS $\rightarrow$ trend-stationary; Holt–Winters MAE < naive MAE.
TEXT	<code>customer_feedback</code> profiled, top-15 unigrams / bigrams + 200-row LSA scatter.
SEGMENTS	Five segment columns (<code>region</code> , <code>employment_type</code> , <code>product_tier</code> , <code>channel</code> , <code>education</code>).
CLEANING	110 duplicates dropped; ~3.5% missing imputed; health gain ~5–10 points.
CHARTS	~20 PNGs in <code>static/charts/</code> .
AI REPORT	Calls out leakage, VIF cluster, imbalance, sensitive columns; structured into 7 markdown sections.
RAG	700–900 facts indexed; <code>search_facts</code> returns top-8 by cosine.

Generated by DataLens AI – comprehensive module reference for `showcase_v3_full.csv`.